

System Architecture and Implementation of an Affine Projection Algorithm (APA) Filter in Simulink

Vishrut Sharma(25210157)

April 4,2026

1 Introduction

This document outlines the architecture, signal flow, and parameterization of a discrete Affine Projection Algorithm (APA) adaptive filter implemented within a custom Simulink subsystem. The model explicitly constructs the matrix operations required for the APA, utilizing Direct Matrix Inversion (DMI) via LU decomposition to compute the adaptive weight updates.

2 System Architecture and Signal Flow

The Simulink subsystem is designed to compute the filter output, error, and coefficient updates using a purely matrix-based signal routing approach. The architecture can be broken down into four distinct computational stages:

2.1 Signal Buffering and Matrix Formulation

The APA requires recent histories of the input data to form the projection matrices.

- **Input Data Matrix (X):** The raw input signal x (Port 1) is first passed through a **6 Delays** block. This delayed signal is then fed into a **Buffer** block to stack the historical samples, forming the $L \times K$ data matrix X .
- **Desired Signal Alignment (d):** To prevent temporal mismatch between the predicted output and the target, the desired signal d (Port 2) is symmetrically delayed using a dedicated **6 Delays** block. This ensures that the target vector spatially aligns with the buffered input matrix X during error computation.

2.2 Filtering and Error Computation

- **A-priori Output Calculation:** The buffered data matrix X is transposed using a **Math Function (u^T)** block. The transposed matrix X^T is then multiplied by the current filter weights $w(n-1)$ using a **Matrix Multiply** block to yield the predicted output vector y .
- **Error Generation:** The time-aligned desired signal vector d and the predicted output vector y are routed into a **Subtract** block (+ -) to generate the error vector $e = d - y$.

2.3 Matrix Inversion and Regularization

The core complexity of the APA lies in inverting the covariance approximation matrix.

- **Covariance Approximation:** The X matrix and its transpose X^T are multiplied to form the $X^T X$ matrix.
- **Tikhonov Regularization:** To prevent singularity and stabilize the inversion, a small regularization factor γ (0.001) is multiplied by an Identity matrix (**eye** block). This regularization term is added to $X^T X$ via an **Add** block.
- **Inversion:** The regularized matrix ($X^T X + \gamma I$) is passed into a **General Inverse** block configured to use LU decomposition.

2.4 Weight Adaptation and Output Extraction

- **Update Term Calculation:** The error vector e is scaled by the adaptation step size μ (**Gain** block set to 0.05). A chain of **Matrix Multiply** blocks computes the final update term: $X \times \text{Inverse} \times (\mu \cdot e)$.
- **Weight Accumulation:** This update term is added to the previous filter weights $w(n-1)$ using a feedback loop containing a unit delay (z^{-1}) block, resulting in the new weights $w(n)$.
- **Scalar Output Selection:** Because the internal operations utilize vectors, **Selector** blocks configured to index 1 ($U(1)$) are applied to the y and e vector lines to extract the most recent scalar output and scalar error for the final subsystem output ports.

3 System Parameters

Based on the implemented subsystem architecture, the APA is parameterized as follows:

Parameter	Value
Delay Line Length (Input x)	6 Samples
Delay Line Length (Desired d)	6 Samples
Adaptation Step Size (μ)	0.05
Regularization Factor (γ)	0.001
Matrix Inversion Method	LU Decomposition
Initial Filter Coefficients	0

The exact mathematical formulation executed by this block diagram is:

$$w(n) = w(n-1) + \mu X(n) [X^T(n)X(n) + \gamma I]^{-1} e(n) \quad (1)$$

4 Implementation Observations

- **Latency Synchronization:** A critical observation during the model’s development was the necessity of time-aligning the d signal with the X matrix. Early iterations lacking the 6 `Delays` block on the d line resulted in a temporal shift on the scope, where the filter output lagged the target due to the inherent buffering latency of X .
- **Dimensional Consistency:** The model relies heavily on strict matrix dimension propagation. Aligning the `Buffer` dimensions, `Delay` blocks, and the `eye` block to a unified configuration ensures that the `General Inverse` block receives a strictly square, non-singular matrix, preventing Simulink compilation errors.